



Gaziantep İslam Bilim ve Teknoloji Üniversitesi
Mühendislik ve Doğa Bilimleri Fakültesi
Bilgisayar Mühendisliği Bölümü

Ödev-1

Ders:

BMB604 || Algoritma Analizi ve Tasarım

Tarih:

[26.05.2026]

Hazırlayan:

[Mehmet ÇELİK - 230007028]

Kullanılan Programlama Dili	Python 3.13
Kodların Bulunduğu Bağlantı	https://drive.google.com/file/d/18ID7K4m8BUC2PfPHg05zrruQePZ870gW/view?usp=sharing
Video Anlatım Bağlantısı	https://drive.google.com/file/d/1X4RZlIIAtZ-EK7wPBnjIrAQASaR9Eh/view?usp=sharing

Danışman:

Dr. Öğr. Üyesi Muhammet Yasin PAK

Mayıs- 2026

Algoritmaların Kısa Açıklaması:

Bubble Sort

Komşu elemanları karşılaştırarak büyük olanı sağa kaydırır. Her geçişte en büyük eleman sona yerleşir. Zaman karmaşıklığı $O(n^2)$, ek bellek gereksinimi $O(1)$ 'dir. Küçük verilerde kullanılabilir fakat büyük verilerde çok yavaştır.

Selection Sort

Her adımda dizinin sıralanmamış kısmındaki en küçük elemanı bularak uygun konuma yerleştirir. Zaman karmaşıklığı $O(n^2)$, ek bellek $O(1)$ 'dir. Takas sayısı Bubble Sort'tan azdır.

Insertion Sort

Elemanları tek tek alarak sıralı alt dizideki doğru konumuna yerleştirir. Zaman karmaşıklığı $O(n^2)$, ek bellek $O(1)$ 'dir. Küçük verilerde ve neredeyse sıralı dizilerde oldukça hızlıdır.

Merge Sort

Diziyi özyinelemeli olarak ikiye böler, alt dizileri ayrı ayrı sıralar, ardından birleştirir. Zaman karmaşıklığı $O(n \log n)$ garantilidir. Ek bellek olarak $O(n)$ geçici alan kullanır. Kararlı (stable) bir algoritmadır.

Quick Sort

Pivot eleman seçerek diziyi ikiye böler: pivottan küçükler sola, büyükler sağa yerleşir. Ortalama zaman karmaşıklığı $O(n \log n)$ 'dir. Ek bellek $O(\log n)$ (yığın). İyi pivot seçimiyle pratikte çok hızlıdır.

Heap Sort

Diziyi önce max-heap veri yapısına dönüştürür, ardından kök elemanı (en büyük) sona alarak heap'i küçültür. Zaman karmaşıklığı $O(n \log n)$, ek bellek $O(1)$ 'dir. In-place ve garantili $O(n \log n)$ sunar.

Radix Sort

Sayıları basamak basamak, en düşük basamaktan başlayarak sıralar. Her basamak için Counting Sort kullanır. Zaman karmaşıklığı $O(d \cdot n)$ 'dir (d =basamak sayısı=11). Ek bellek $O(n+k)$ kullanır.

Test Dizileri ve Ölçüm Yöntemi:

Test Dizileri

11 basamaklı rastgele tam sayılar kullanılmıştır (10.000.000.000 – 99.999.999.999 aralığı).

Test Boyutları:

- $O(n^2)$ algoritmalar için: 100, 500, 1.000, 2.000, 5.000
- $O(n \log n)$ algoritmalar için: 100, 500, 1.000, 2.000, 5.000, 10.000, 50.000, 100.000

Her algoritma **aynı veri seti** üzerinde test edilmiştir. Bir algoritma çalıştırılmadan önce orijinal dizinin kopyası alınmış, böylece tüm algoritmalar aynı başlangıç verisini sıralamıştır.

Çalışma Süresi Ölçümü

```
###python
import time
start = time.perf\_counter()
sort\_function(arr)
elapsed = time.perf\_counter() - start
```

`time.perf\\_counter()` yüksek çözünürlüklü sistem saatini kullanır.

Bellek Kullanımı Ölçümü

```
###python
import tracemalloc
tracemalloc.start()
sort\_function(arr)
current, peak = tracemalloc.get\_\_traced\_\_memory()
tracemalloc.stop()
peak\_kb = peak / 1024
```

`tracemalloc` modülü ile tepe (peak) bellek kullanımı KB cinsinden ölçülmüştür.

Sonuç Tablosu

Algoritma	Veri Boyutu	Çalışma Süresi (s)	Bellek Kullanımı (KB)
Bubble Sort	100	0.000498	0.11
Bubble Sort	500	0.049447	0.25
Bubble Sort	1.000	0.357480	0.26
Bubble Sort	2.000	1.884298	0.26
Bubble Sort	5.000	12.588441	0.26
Selection Sort	100	0.000196	0.11
Selection Sort	500	0.031086	0.23
Selection Sort	1.000	0.153183	0.26
Selection Sort	2.000	0.646496	0.26
Selection Sort	5.000	4.116930	0.26
Insertion Sort	100	0.000141	0.08
Insertion Sort	500	0.021601	0.14
Insertion Sort	1.000	0.147129	0.14
Insertion Sort	2.000	0.697506	0.14
Insertion Sort	5.000	4.555581	0.14
Merge Sort	100	0.000182	1.73
Merge Sort	500	0.001029	8.03
Merge Sort	1.000	0.003716	15.77
Merge Sort	2.000	0.011845	31.42
Merge Sort	5.000	0.048481	78.50
Merge Sort	10.000	0.121949	156.55
Merge Sort	50.000	0.840034	781.73
Merge Sort	100.000	1.936606	1562.90
Quick Sort	100	0.000059	0.08
Quick Sort	500	0.001912	0.67
Quick Sort	1.000	0.006822	0.92
Quick Sort	2.000	0.017074	1.05
Quick Sort	5.000	0.052718	1.30

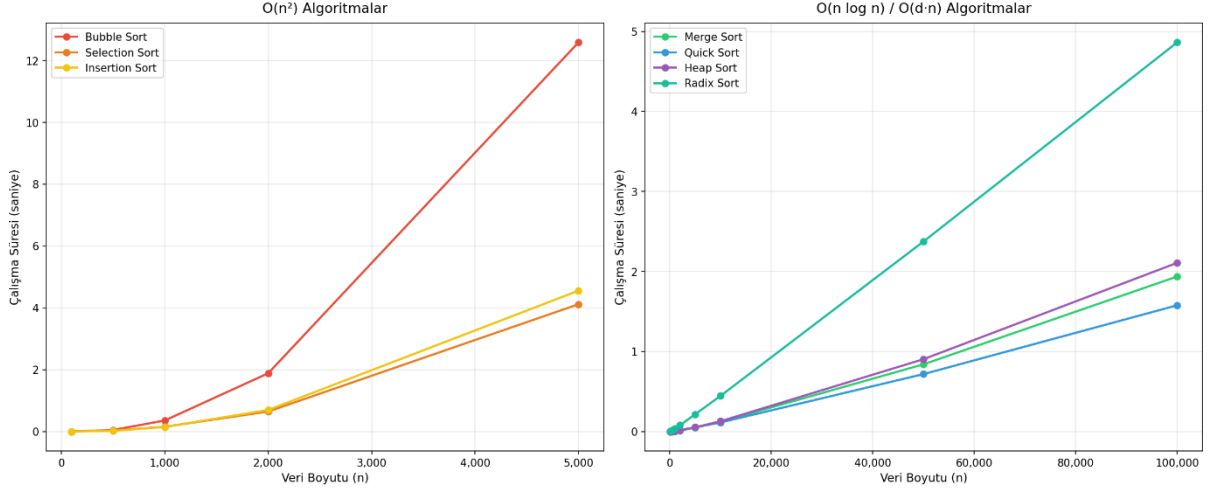
Quick Sort	10.000	0.113387	1.36
Quick Sort	50.000	0.717558	1.86
Quick Sort	100.000	1.577141	1.80
Heap Sort	100	0.000106	0.08
Heap Sort	500	0.001918	0.25
Heap Sort	1.000	0.006075	0.31
Heap Sort	2.000	0.016269	0.37
Heap Sort	5.000	0.052712	0.50
Heap Sort	10.000	0.128751	0.56
Heap Sort	50.000	0.903889	0.68
Heap Sort	100.000	2.107701	0.75
Radix Sort	100	0.001722	1.03
Radix Sort	500	0.015500	4.37
Radix Sort	1.000	0.034341	8.34
Radix Sort	2.000	0.078743	16.18
Radix Sort	5.000	0.216425	39.65
Radix Sort	10.000	0.445952	78.71
Radix Sort	50.000	2.374791	391.21
Radix Sort	100.000	4.862554	781.84

> **Not:** Bubble Sort, Selection Sort ve Insertion Sort $O(n^2)$ karmaşıklığa sahip olduğundan

> **n > 5.000** için test edilmemiştir; bu boyutlarda çalışma süreleri kabul edilemez uzunluğa ulaşmaktadır.

Grafikler:

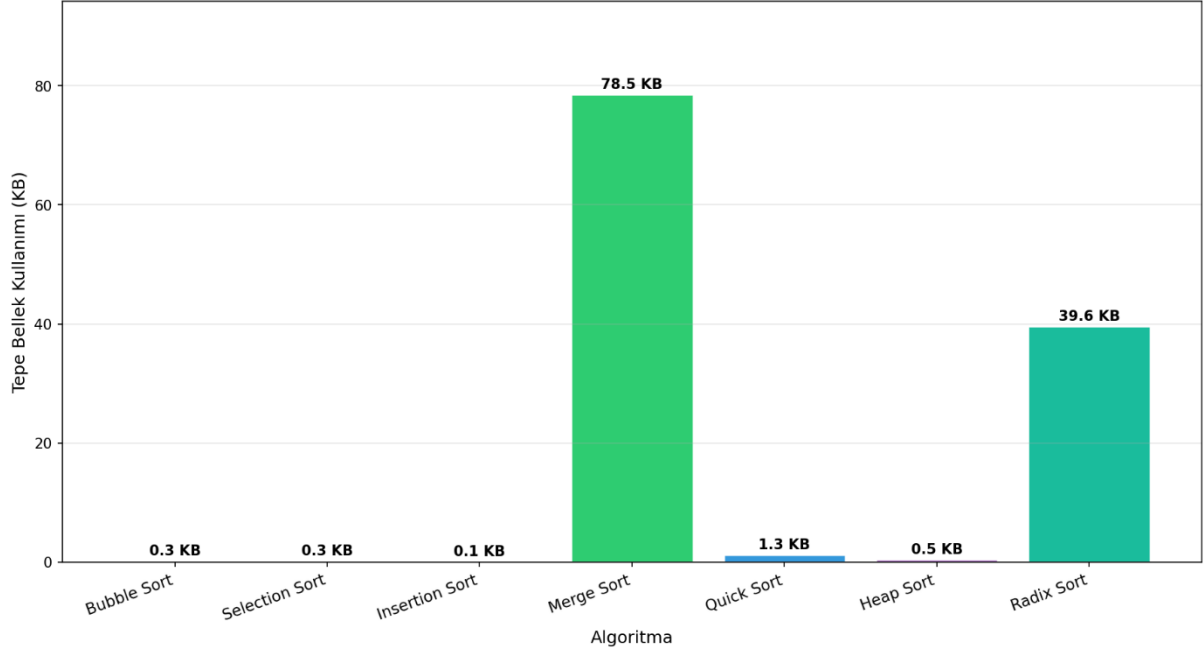
Veri Boyutuna Göre Çalışma Süresi Karşılaştırması



Veri boyutuna göre çalışma süresi (Grafik 1)

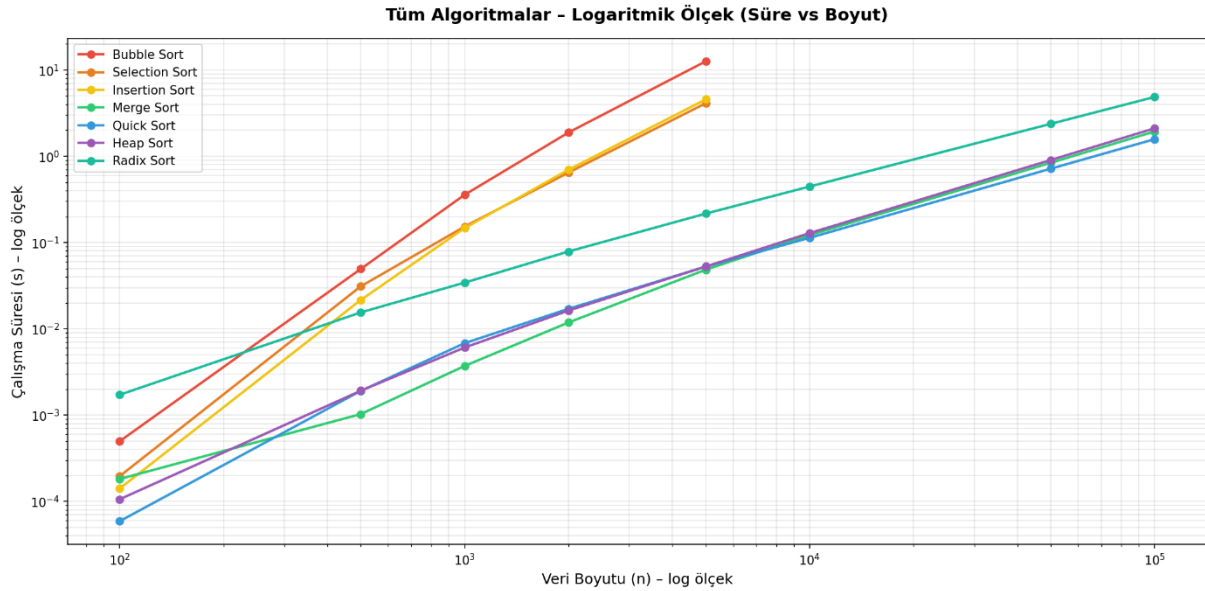
>> Grafik 1 incelendiğinde $O(n^2)$ algoritmaların veri boyutu arttıkça üstel biçimde yavaşladığı görülmektedir. $O(n \log n)$ algoritmalar ise veri boyutuna rağmen çok daha düşük sürede tamamlanmaktadır. $n=5000$ 'de Bubble Sort 12.6 saniye harcarken Merge Sort yalnızca 0.048 saniye almıştır.

Algoritmalarla Göre Bellek Kullanımı (n = 5,000)



Algoritmalarla göre bellek kullanımı (Grafik 2)

>> Grafik 2 incelendiğinde Bellek kullanımı açısından Bubble Sort, Selection Sort, Insertion Sort ve Heap Sort neredeyse sıfır ek bellek kullanırken, Merge Sort $O(n)$ ek bellek gerektirdiğinden en yüksek tepe değerine ulaşmıştır. Radix Sort ise Merge Sort'a yakın bellek harcamaktadır.



Logaritmik ölçek karşılaştırma (ek grafik)

Karşılaştırma ve Analiz:

Hangi algoritma küçük veri boyutlarında daha iyi sonuç vermiştir?

> Küçük veri boyutlarında ($n=100$) Quick Sort 0.000059 saniye ile en hızlı sonucu vermiştir. Insertion Sort ve Heap Sort da bu boyutta çok yakın değerlere ulaşmıştır. Küçük dizilerde $O(n^2)$ algoritmaların teorik dezavantajı henüz belirgin olmadığından, sabit faktörü düşük olan algoritmalar öne geçmiştir.

Büyük veri boyutlarında hangi algoritmalar daha avantajlıdır?

> $n=100.000$ için Quick Sort 1.577 saniye ile en hızlı, onu sırasıyla Merge Sort (1.937 s) ve Heap Sort (2.108 s) takip etmiştir. $O(n \log n)$ garantili bu algoritmalar büyük verilerde açık ara öne çıkmaktadır. Radix Sort Python'da döngü yavaşlığı nedeniyle daha yavaş kalmıştır.

$O(n^2)$ algoritmaların veri boyutu arttıkça performansı nasıl değişmiştir?

> $O(n^2)$ algoritmalar veri boyutu 2 katına çıktığında çalışma süreleri yaklaşık 4-5 kat artmıştır. Bubble Sort $n=1000$ 'de 0.36 saniye iken $n=5000$ 'de 12.59 saniyeye ulaşmıştır. Bu, $O(n^2)$ büyümeyi açıkça doğrulamaktadır. Bu nedenle $n > 5.000$ için test kapsamı dışı bırakılmıştır.

Merge Sort, Quick Sort ve Radix Sort arasında nasıl bir fark gözlenmiştir?

> Quick Sort pratikte en hızlı ve en az bellek kullanan (1.80 KB, $n=100.000$) algoritma olmuştur. Merge Sort garantili $O(n \log n)$ sunar ve kararlıdır; ancak 1562 KB ek bellek kullanır. Radix Sort teorik olarak $O(d \cdot n)$ ile en verimli olmalıdır fakat Python'daki döngü yükü nedeniyle en yavaş sonucu vermiştir.

Bellek kullanımı açısından hangi algoritmalar daha avantajlıdır?

> In-place algoritmalar (Bubble, Selection, Insertion, Heap Sort) neredeyse sıfır ek bellek kullanır. Quick Sort yalnızca rekürsif çağrı yığını için küçük miktarda bellek harcar ($n=100.000$ 'de 1.80 KB). Merge Sort ve Radix Sort ise $O(n)$ düzeyinde ek bellek gerektirir. Bellek kısıtlı sistemlerde Heap Sort en uygun seçimdir: hem $O(n \log n)$ hem de $O(1)$ ek bellek.

Deneyisel sonuçlar teorik zaman karmaşıklıklarıyla uyumlu mudur?

> Evet, büyük ölçüde uyumludur. $O(n^2)$ algoritmalar veri boyutu 2 katına çıkınca süre yaklaşık 4x artmış, $O(n \log n)$ algoritmalar ise veri boyutu 10 katına çıkınca süre yaklaşık 13-16 kat artmıştır (teorik beklenti $\sim 13.3x$). Küçük sapmalar Python yorumlayıcısının ek maliyeti, CPU önbellek etkileri ve işletim sistemi zamanlama farklılıklarından kaynaklanmaktadır.